

A guide for checking the paper “Pattern-avoiding binary arrays and a Stirling-type table”

This note accompanies the 37-page version of 10 October 2026. Page numbers refer to that version. The repository is <https://github.com/r5x8ksva-del/a207123-explicit-formulas>.

1. What the paper claims

- **Structure for fixed m** (Theorems A and B; Theorems 3.1 and 4.6): the minimal order of a linear recurrence in k for $U_k(m)$ is exactly $3m + 1$, and $U_k(m)$ has a positive double-sum formula in Stirling and r -Stirling numbers.
- **All recurrences** (Theorem C; Theorems 5.1, 5.4, B.1, Proposition 5.6, Corollary 5.7): the generating function is not D-finite; the operators annihilating U on a quadrant form the left ideal $\mathcal{OL}_1$, and those annihilating the triangle N form $\mathcal{OL}_N$, with exact dimension counts; U is not Stirling-like in the sense of Kauers.
- **Real roots** (Theorem D; Theorem 8.1, Corollaries 8.2 and 8.13): the polynomials h_k have only real, simple zeros, $\lfloor k/3 \rfloor$ of them greater than 1; equivalently the chain polynomials of the posets Λ_k are real-rooted.
- **Further results:** obstructions to specific single-sum formulas (Theorem E, Section 6) and the near-diagonal of the triangle (Theorem F, Section 7).

The paragraph *What is new* on p. 5 states which tools are standard and which steps are specific to these arrays.

2. What is machine-checked

The results in Table 5 (p. 30) are proved in Lean 4 with Mathlib from the definitions of $a_k(n)$ and $U_k(m)$: the reduction, the basic recurrence, the triangle N , the exact orders (Theorem 3.1), the roots in Theorem 3.2(1), Corollary 3.4, the block decomposition (Theorem 4.2), the generating function in x and y and the coefficient formula (Theorem 4.4, Lemma 4.5) and the explicit formula (Theorem 4.6), through a refined recurrence, the non-D-finiteness (Theorem 5.1), the ${}_1F_1$ form of $\sum_m t^m/P_m$ (Remark 5.2), the descriptions $\text{Rel}(U) = \mathcal{OL}_1$ and $\text{Rel}(N) = \mathcal{OL}_N$ with the dimension formulas (Theorems 5.4 and B.1), Corollary 5.5 (with denominators cleared), Proposition 5.6, Corollary 5.7 (with denominators cleared), and the near-diagonal theorem (Theorem 7.1, Proposition 7.2), Corollary 8.15 and part of Lemma 8.12. The axiom check reports only `propext`, `Classical.choice` and `Quot.sound`. Lean checks the proofs, not whether the formal statements say what the paper says: **Appendix A (pp. 30–33) transcribes the definitions that carry the meaning**, and comparing them with the paper is a short and useful check.

3. Where a human check is most needed

The proofs were written with substantial assistance from an AI model and cross-checked by further AI runs (see the statement at the end of the paper). The parts below are not formalized, so an independent reading by an expert is the main remaining safeguard. In order of priority:

1. **Section 8, real-rootedness (pp. 24–29).** Lemmas 8.4–8.7 collect standard interlacing facts, and Lemma 8.8 applies them to the operators $\mathcal{D}$, $\mathcal{T}$ and Ψ of the recurrence. The core is Lemma 8.9 (the recurrence for the row polynomials), Proposition 8.10 (four interlacing relations propagate along the recurrence), Proposition 8.11 (simplicity of the zeros) and Lemma 8.12 (the order of the zero -1 and the sign of the cofactor there; of this lemma only a_k , $\deg h_k$ and the sign of the leading coefficient of h_k are formalized); Corollaries 8.13 and 8.14 are consequences; Corollary 8.15 is formalized. Remark 8.16 explains why the general theorems on chain polynomials do not apply: the zeros are not confined to $[-1, 0]$.
2. **Section 6, obstructions (pp. 16–22).** Theorems 6.1–6.3 use residues on fibres and norms in number fields. Theorem 6.5 (two single sums) is computer-assisted: a sieve modulo 5040 and Skolem’s p -adic method (Propositions 6.8–6.10, Lemma 6.11); the scripts are in the repository.
3. **Section 4, block decomposition (pp. 10–12):** the identification with r -Stirling numbers in Lemma 4.5 and the bijection of Proposition 4.7 (the formal proofs of Theorems 4.4 and 4.6 avoid them).
4. **Section 5:** Remark 5.8, the step from Kauers’ definition of Stirling-like sequences to the formalized Corollary 5.7, and the choice of a common denominator in Corollaries 5.5 and 5.7.
5. **Section 3:** parts (2) and (3) of Theorem 3.2 (asymptotics) and Remark 3.3.

Appendix B (pp. 33–36) is formalized, but its written proof is new; comments on its readability are welcome.

4. Questions on novelty

Searches in arXiv, OpenAlex, Crossref and Google Scholar found no previous work on these arrays and no precedent for the two points below; MathSciNet, zbMATH and Web of Science have not been searched yet. An expert’s knowledge would help most here: (a) Is real-rootedness known for chain polynomials of a class of posets containing Λ_k , or for chain polynomials with zeros outside $[-1, 0]$? (b) Has the annihilator ideal of a bivariate sequence been determined for a generator that is not of Stirling-like shape, as in Theorems 5.4 and B.1?

5. Reproducing the computations

From the repository root (Python 3; only `verify_all.py` needs `numpy`):

- `python3 verify_all.py` — 414 automated checks of the results and numbers, about 11 minutes;
- `python3 code/main_extra/check_paper_numbers.py paper/main.tex` — recomputes the numbers printed in the paper (32 checks);
- `python3 code/main_extra/check_appendix_lean.py paper/main.tex` — compares Appendix A with the Lean sources (35 declarations);
- `python3 code/main_extra/check_appendix_relN.py` — exact checks of the operator identities of Lemma B.2 (14 checks);
- `python3 code/main_extra/check_chains_negzeros.py` — Corollary 8.15 and Remark 8.16 (6 checks).

Lean: `toolchain leanprover/lean4:v4.34.1` with `Mathlib v4.34.1` (see `lean/lean-toolchain` and `lean/lake-manifest.json`). The usual `Mathlib` workflow in `lean/` (`lake exe cache get`, then `lake build`) should apply, but the author has built the development only on Windows, with `lean/run_lean_checks_direct.sh`; a single module needs up to about 8 GB of memory. `lake env lean Axioms.lean` prints the axioms used.